Replicación de *Feature Visualization*

Los ResNet de Pl@ntNet-300K y una herramienta para inspeccionarlos

David Fernández Fernández y Javier Santamaría Pascual

1. Introducción

Replicamos *Feature Visualization*, de Olah, Mordvintsev y Schubert (2017), sobre los ResNet preentrenados en Pl@ntNet-300K –el de 18 capas como modelo de referencia, y los de 50 y 152 para comprobar que las técnicas no dependen de la arquitectura–, y construimos encima una herramienta de inspección: se le da una foto de una planta y responde qué especie ve el modelo, qué neuronas enciende esa foto, en qué zona las enciende y qué patrón busca cada una de ellas.

El artículo no publica ninguna métrica. Es un trabajo cualitativo: su resultado son figuras, no cifras, de modo que no hay un número que igualar. Lo que se replica son sus **técnicas** y sus **figuras**, y lo que sí admite medida es el argumento que defiende, que sí es comprobable (sección 4).

Tabla 1: Alcance de la replicación.

Técnica del artículo	**Repr.**	**Dónde**
Objetivos: Neuron, Channel, Layer/DeepDream, Class Logits, Class Probability	sí	`objetivos.py`
Direction, suma de neuronas, interpolación	sí	«Interaction between Neurons»
Diversity	sí	`generar_diversas()`
Preconditioning: Fourier con color decorrelado	sí	contrastado contra `lucid`
Transformation robustness	sí	valores de la nota 9 del artículo
Penalización de frecuencia: L1, variación total, desenfoque	sí	fórmulas de `lucid`
Dataset examples	sí	`ejemplos_del_dataset()`
Learned priors (GAN/VAE)	**no**	fuera de alcance: exige entrenar un generativo

2. El artículo en resumen

Una red entrenada es un montón de números y no dice qué ha aprendido. La visualización de características lo plantea al revés: en lugar de preguntar qué responde la red ante una foto, busca **qué imagen haría responder al máximo** a una unidad concreta. Se parte de ruido y se asciende por el gradiente hasta que esa unidad se activa todo lo posible.

El punto clave, y lo que distingue esta técnica del entrenamiento, es que **los pesos del modelo no se tocan**. Se cargan congelados, y lo único que recibe gradiente —y por tanto lo único que se optimiza— es la imagen de entrada.

Hecho así de ingenuo el método no funciona: produce ruido adversarial, una imagen que activa muchísimo la unidad y no se parece a nada reconocible. El artículo sostiene que la diferencia entre ruido y patrón interpretable está en la regularización, y describe tres familias: penalizaciones de frecuencia sobre la imagen, robustez a transformaciones, y *learned priors*. Reproducimos las dos primeras.

2.1. Qué se puede maximizar

Tabla 2: Los ocho objetivos implementados. Los cinco primeros son los de «Optimization Objectives»; los tres últimos, los de «The Interaction between Neurons».

Tipo	Expresión	Qué maximiza
`canal`	`layer_n[:,:,z]`	la media espacial de un canal; es el valor por defecto
`neurona`	`layer_n[x,y,z]`	una sola posición de ese canal
`capa`	`layer_n[:,:,:]`^{2}	la capa entera: es DeepDream, e ignora el canal
`logit`	`pre_softmax[k]`	la puntuación de una clase
`probabilidad`	`softmax[k]`	esa clase, hundiendo además las demás
`direccion`	$\mathbf{v} \cdot$ `layer_n[x,y,:]`	una combinación de canales, no uno solo
`suma`	varios canales a la vez	el objetivo *jointly optimized*
`interpolacion`	de un canal a otro	la transición entre dos unidades

3. Arquitectura y funcionamiento

El sistema tiene dos mitades que no se mezclan. La de replicación contiene los objetivos, el bucle de optimización y la fachada; la del aplicativo es una interfaz web que solo carga un *checkpoint* y responde. El aplicativo no importa nada del código de entrenamiento: habla únicamente con la fachada, que es el contrato.

3.1. Los modelos

Pl@ntNet-300K publica tres *checkpoints* y el sistema admite los tres, sobre las mismas 1081 especies. Cambiar de uno a otro es cambiar dos claves del fichero de configuración: no hay código específico de ninguna arquitectura, y las cuatro capas que ofrece la interfaz (`layer1.1.conv2` a `layer4.1.conv2`) existen en los tres.

Arquitectura	Parámetros	Papel en el trabajo
ResNet-18	11 731 065	El de referencia. Todas las cifras y figuras de este informe salen de él.
ResNet-50	25 723 001	Mismas técnicas, unidades más anchas.
ResNet-152	60 358 777	El más profundo de los tres.

Que las técnicas no dependan de la arquitectura es, precisamente, parte de lo que el artículo sostiene: lo que describe son métodos para interrogar una red convolucional cualquiera, no recetas atadas a un modelo concreto. Poder ejecutarlos sobre tres redes de tamaño muy distinto es la forma de comprobarlo.

3.2. El recorrido de la aplicación

Las etapas 2 y 3 son el mismo cálculo visto de dos maneras, y conviene entender por qué. Un *hook* captura la salida de la capa elegida, que no es un número por canal sino una rejilla: 56×56 en `layer1.1.conv2`, 28×28 en `layer2`, 14×14 en `layer3` y 7×7 en `layer4`. El ranking promedia esa rejilla para poder ordenar los canales —ese promedio es exactamente el objetivo *Channel* del artículo, medido en lugar de maximizado— y el mapa de activación devuelve la misma rejilla **antes**

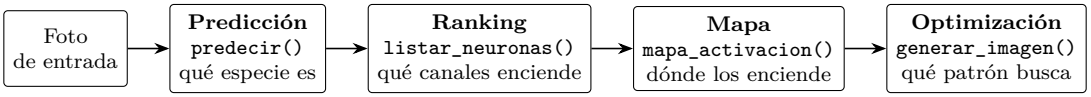


Figura 1: Las cuatro etapas. Son las mismas desde la interfaz web y desde la API en Python, porque ambas llaman a las mismas funciones de la fachada.

del promedio, reescalada al tamaño de la foto. El mapa es, por tanto, el eslabón que conecta un número del ranking con una zona concreta de la imagen.

3.3. El bucle de optimización

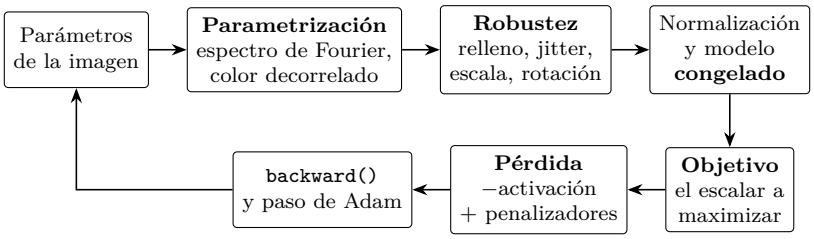


Figura 2: Una iteración. El gradiente llega hasta los parámetros de la imagen y Adam los actualiza; los pesos del modelo quedan intactos.

Dos piezas de la figura 2 son las que el artículo defiende y las que marcan la diferencia entre un patrón y ruido. La **parametrización** no optimiza los píxeles directamente, sino un espectro de Fourier escalado por $1/f^{\,\mathrm{decay}}$ con decorrelación de color y una sigmoide: es el *preconditioning*, y lo que hace es cambiar qué direcciones del espacio de imágenes son «baratas» para el optimizador. La **robustez** encadena relleno, desplazamiento aleatorio, escalado, rotación, un segundo desplazamiento y recorte, de modo que solo sobreviven los patrones que activan la unidad aunque la imagen se mueva.

Si en los primeros pasos el gradiente es exactamente cero, se lanza la excepción `UnidadMuerta`. Es el caso de una unidad ReLU muerta: no da error por sí sola, simplemente devolvería el ruido inicial sin avisar de nada.

3.4. Configuración por capas

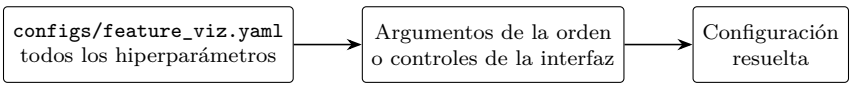


Figura 3: Ningún hiperparámetro está escrito a fuego. Lo que se pasa por fuera sobrescribe la clave correspondiente sin modificar el fichero, así que el estado por defecto siempre es el documentado.

El fichero se reparte en bloques para el objetivo, la imagen, la optimización, la regularización, la robustez, la normalización, la salida y el registro de experimentos. La interfaz expone nueve de esos parámetros como controles —los pasos de optimización, los tres penalizadores de frecuencia, el desplazamiento, la rotación, la escala y los dos interruptores que apagan la robustez y el espacio de Fourier—, que son precisamente los de las figuras interactivas del artículo. A ellos se suman el selector de capa y el de objetivo.

Los dos interruptores son los que mejor ilustran el argumento central: al apagarlos la activación final sube y la imagen deja de parecerse a nada.

Tabla 3: Qué hay en cada sitio.

Carpeta o archivo	Responsabilidad
`src/interpretability/objetivos.py`	Qué se maximiza: los ocho objetivos del artículo.
`src/interpretability/feature_viz.py`	El bucle, la parametrización y la robustez. También es la orden de consola.
`src/interpretability/api.py`	La fachada: el único contrato que consume el aplicativo.
`src/app/serve.py`	La interfaz web. No importa nada de `src/training/`.
`src/utils/`	Semillas, registro de experimentos y utilidades.
`configs/`	Todos los hiperparámetros.
`scripts/`	Descarga de datos y pesos, medida del acierto y las dos demostraciones.
`tests/`	133 pruebas de forma, invariancia y reproducibilidad.

3.5. Organización del repositorio

Las semillas se fijan en un único módulo y las dependencias van con versión exacta; entre ejecuciones distintas la activación final puede variar en torno a un 1 %. Las pruebas no dependen de los pesos: usan un ResNet-18 sin entrenar, porque los `.tar` no se versionan. Cada ejecución queda registrada en Weights & Biases con la activación, la pérdida, la aspereza de la imagen y la imagen en curso cada 32 pasos, que es la figura *Step 1 / 32 / 128 / 256* del artículo generada automáticamente.

4. Resultados



Figura 4: El recorrido completo sobre una misma foto. Izquierda: la entrada, clasificada como *Lactuca serriola* con un 63,0 % de probabilidad. Centro: el mapa de activación del canal 362 de `layer4.1.conv2`, el más activo de esa capa (+0,3392). Derecha: la imagen optimizada desde ruido para maximizar ese mismo canal.

4.1. Verificación del *checkpoint*

Antes de interpretar un modelo hay que asegurarse de que se está interpretando el modelo correcto, y ese riesgo aquí es real: un error en la normalización de entrada o en el mapa de clases **no produce ningún fallo visible**. El programa sigue funcionando y las visualizaciones salen igual de vistosas, solo que entonces son de otra unidad, o de un modelo mal cargado, y todo lo que se concluya de ellas es falso. Medir el acierto es la forma de detectarlo, porque se desploma en cuanto algo de eso está mal.

Sobre 120 fotos del *split* de test, una por especie, entre las 144 especies con 300 o más imágenes de entrenamiento, el clasificador acierta **80,0 % de top-1 y 95,0 % de top-5**. Son cifras coherentes con un ResNet-18 sobre 1081 clases, de modo que el *checkpoint* se está cargando y usando como debe y las visualizaciones son de lo que decimos.

La misma medida resolvió además una ambigüedad que el *checkpoint* no documenta: con qué estadísticas se normalizó. Las de ImageNet dan 80,0 % y las de Pl@ntNet-300K 79,2 %, con el top-5 idéntico. Una imagen de diferencia en 120, así que la elección no cambia el resultado y deja de ser una duda abierta.

4.2. El argumento central, cuantificado

El artículo sostiene que sin robustez a transformaciones la optimización alcanza un óptimo mejor y una imagen peor. Es lo único verdaderamente medible de la replicación, y se comprueba desactivándola.

Tabla 4: Ablación de la robustez. Canal 181 de `layer3.1.conv2`, 256 pasos. La aspereza es la variación total de la imagen, que mide cuánto se parece a ruido: cuanto más alta, peor.

	Activación final	**Aspereza**
Con robustez	+4,93	0,164
Sin robustez	+7,15	0,245
Diferencia	+45 %	+49 %

La activación sube un 45 % mientras la imagen empeora un 49 %. Mejor óptimo, peor imagen: exactamente lo que el artículo defiende. Apagar el espacio de Fourier produce el mismo efecto, con convergencia a ruido adversarial.

5. Diferencias con el artículo original

Referencias

[1] C. Olah, A. Mordvintsev y L. Schubert. *Feature Visualization*. Distill, 2017. DOI: 10.23915/distill.00007.

[2] C. Garcin, A. Joly, P. Bonnet et al. *Pl@ntNet-300K: a plant image dataset with high label ambiguity and a long-tailed distribution*. NeurIPS Datasets and Benchmarks, 2021.

[3] *tensorflow/lucid*. Implementación de referencia del artículo, en TensorFlow.

Artículo original	**Nuestro proyecto**	**Motivo**
GoogLeNet entrenada en ImageNet.	Los tres ResNet de Pl@ntNet-300K (18, 50 y 152 capas), 1081 especies.	Es el dominio de la asignatura. Ejecutar las mismas técnicas sobre tres redes de tamaño muy distinto comprueba que no dependen de la arquitectura.
2560 pasos de optimización por figura.	256 por defecto, ajustable entre 32 y 1024 desde la interfaz, y sin límite desde el fichero de configuración.	Coste por imagen. Con 256 las figuras salen algo menos refinadas; subir ese único parámetro a 2560 recupera la fidelidad del artículo sin tocar una línea de código.
Valores de robustez de la nota al pie 9.	Los mismos de la nota al pie 9.	Para una replicación manda el artículo. `lucid`, su propia biblioteca, usa otros valores en `standard_transforms`.
Coeficientes L1 $-0{,}05$, TV $-0{,}25$, desenfoque -1.	No se copian literalmente.	Allí los términos se **suman** sobre 150 528 elementos y aquí se **promedian**; además allí se maximiza y se restan, y aquí se minimiza y se suman.
Tres familias de regularización, *learned priors* incluidos.	Las dos primeras.	Los *learned priors* exigen entrenar un modelo generativo, que es un proyecto aparte.